



UNIVERSITÀ DEGLI STUDI DI SALERNO

Corso di Fondamenti di Intelligenza Artificiale (FIA)

Anno Accademico 2025/26



HAIVE

an AI for the bug-themed strategy game

15 febbraio 2026

Autore	Matricola
Forte Guglielmo	0512119526
Rega Maristella	0512119032
Russo Serena	0512119098
Squitieri Andrea	0512119008

LINK GITHUB: <https://github.com/andrea-sq/hAIve>

Indice

1	Introduzione	2
2	Descrizione del problema	3
2.1	Obiettivi	3
2.2	Specifiche PEAS	3
2.2.1	Caratteristiche dell'ambiente	3
2.3	Analisi del problema	4
2.3.1	Rappresentazione dello stato di gioco	5
2.3.2	Contributi precedenti	5
3	Soluzioni proposte	6
3.1	Esplorazione mosse	6
3.1.1	Potatura alfa-beta	6
3.1.2	Monte Carlo Tree Search	8
3.2	Valutazione stato di gioco	9
3.2.1	Funzioni euristiche	9
3.2.2	NNUE	10
4	Confronto tra le implementazioni	10

1 Introduzione

Hive è un gioco da tavolo strategico per due giocatori, ideato da John Yanni e pubblicato nella prima edizione nel 2001.

Rientra nei giochi deterministici ad informazione perfetta e a somma zero, tuttavia, a differenza di esempi più celebri come il gioco degli scacchi, non possiede una plancia e può essere svolto su qualsiasi superficie piana.

In seguito alla più recente espansione, il numero totale di pezzi è 28, di cui 14 bianchi e 14 neri, raffiguranti un insieme di insetti, aracnidi e isopodi, ognuno con un modo particolare di muoversi. L'obiettivo è catturare l'*ape regina* avversaria, facendo sì che sia completamente circondata da altri pezzi.

Perché Hive? Un'analisi del branching-factor [ai_for_hive] mostra che, negli stadi più avanzati del gioco, il fattore di ramificazione è stabile a 60. Pertanto, la crescita esponenziale di Hive è circa due volte quella degli scacchi, il cui fattore di ramificazione è intorno al 30.

L'interesse nel creare un'intelligenza artificiale in grado di giocare ad Hive non deriva solo dall'elevata complessità, ma anche dalla scarsa notorietà del gioco che lo rende un problema non ancora esplorato esaustivamente.

2 Descrizione del problema

2.1 Obiettivi

Lo scopo del progetto consiste nella realizzazione di un **modello di intelligenza artificiale** in grado di sfidare un altro agente intelligente — artificiale o umano — nel gioco di *Hive*.

2.2 Specifiche PEAS

- **Performance.** Le misure adottate per valutare le prestazioni sono il *tempo* per generare una mossa e le *percentuali di vittoria* contro una generatore di mosse randomiche e altri agenti noti.
- **Environment.** L'ambiente in cui opera l'agente è costituito dall'insieme di tutte le possibili configurazioni che l'alveare può assumere.
- **Actuators.** L'agente agisce sull'ambiente mediante la successiva mossa scelta.
- **Sensors.** L'agente percepisce l'ambiente mediante la sequenza di mosse dell'avversario.

2.2.1 Caratteristiche dell'ambiente

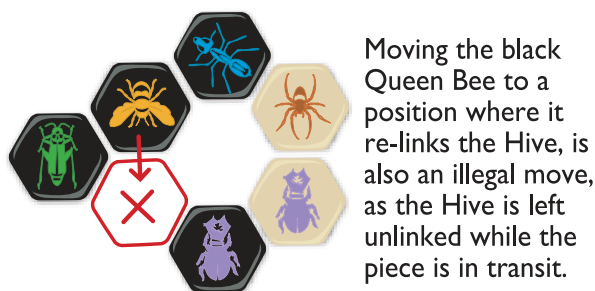
L'ambiente operativo è:

- **Completamente osservabile.** I sensori forniscono accesso allo stato completo dell'ambiente in ogni momento.
- **Deterministico.** Lo stato successivo è completamente determinato da stato corrente e dalla seguente azione scelta.
- **Sequenziale.** Ogni decisione può influenzare tutte quelle successive.
- **Statico.** L'ambiente rimane invariato mentre l'agente delibera.
- **Discreto.** Il numero di percezioni e azioni distinte è finito.
- **Multiagente competitivo** L'ambiente consente la presenza di esattamente due agenti, ognuno dei quali tenta di sconfiggere l'avversario.

2.3 Analisi del problema

Il gioco è costituito da poche e semplici regole, di seguito riassunte per facilitare la comprensibilità del problema.

- Ad ogni turno un giocatore potrà eseguire una delle due possibili mosse:
 1. Far entrare in gioco un insetto (finché ne restano a disposizione);
 2. Muovere un insetto già posizionato (se in grado di farlo).
- Facendo entrare in gioco un insetto, lo si deve posizionare in modo che nessuno dei suoi lati tocchi un insetto del colore avversario già posizionato, ma unicamente insetti del proprio colore. (Fa eccezione la prima mossa di ogni giocatore).
- L'ape regina deve entrare in gioco entro il quarto turno e, in mancanza di essa, i pezzi in gioco non possono muoversi.
- Se un giocatore non può muovere un suo pezzo in campo né aggiungerne di nuovi, passa il turno all'avversario: il gioco continua in questo modo finché il primo giocatore non è nuovamente in grado di fare la sua mossa.
- *One Hive Rule*: i pezzi in gioco devono essere costantemente connessi, non è mai possibile dividere l'alveare in due o più parti.



- *Freedom To Move*: poiché le creature si muovono scorrendo intorno all'alveare, un pezzo può muoversi da o verso una posizione solo se può fisicamente scivolare in quella direzione. Fa eccezione a questa regola il Grillo, ed eventualmente la Zanzara quando lo imita, in quanto può saltare sopra l'alveare. La Coccinella e lo Scarabeo sono in grado di spostarsi da o verso posizioni totalmente circondate muovendosi sopra l'alveare, ma devono comunque rispettare la regola nel caso al secondo livello vi sia un "varco stretto".



2.3.1 Rappresentazione dello stato di gioco

Per modellare lo stato in cui si trova l'alveare sono state adottate due rappresentazioni distinte:

- Una mappa esagonale modulare di raggio 14 (e diametro 29) a cui abbiamo applicato le coordinate assiali per identificare univocamente la posizione di ogni esagono. La scelta del raggio corrisponde al più piccolo numero che permette di posizionare tutti i 28 pezzi e mantenere l'illusione di una mappa infinita, in modo da minimizzare la quantità di spazio necessario a memorizzare ogni stato.
- Un vettore binario che rappresenta la presenza o l'assenza di possibili archi e l'altezza dei pezzi che sono in grado di salire sull'alveare.

2.3.2 Contributi precedenti

Il progetto Mzinga [**mzinga**] è una collezione di software open-source per agevolare la comunità di sviluppatori e giocatori di Hive.

Offre tra le principali feature l'**Universal Hive Protocol** (UHP) che supporta l'interoperabilità tra software dedicati al gioco. Opera distinguendo tra due principali tipi di programmi: engines e viewers. Un engine ha il compito di comprendere come funziona una partita di Hive. Un viewer semplifica l'interazione dell'utente umano con l'engine.

Abbiamo deciso di implementare **UHP** per semplificare l'interazione e il confronto tra il nostro agente e altri agenti pre-esistenti. Ulteriore beneficio di questa scelta è derivato dall'utilizzo del viewer di Mzinga, che ci ha fornito un'interfaccia di gioco ampiamente utilizzata dalla community.

3 Soluzioni proposte

A partire dall'analisi effettuata, risulta naturale che si tratti di un problema di ricerca con avversario.

Per giungere ad una soluzione è necessario definire due componenti:

- **Algoritmo di esplorazione delle mosse.** Per ricercare la scelta più conveniente, non è sufficiente conoscere le mosse possibili in un dato istante, ma è necessario studiare le implicazioni di queste e quindi le possibili mosse n turni nel futuro.
- **Algoritmo di valutazione dello stato di gioco.** È impensabile esplorare completamente l'albero di gioco a causa dell'elevato *fattore di ramificazione*, che ricordiamo essere pari a 60. Di conseguenza, è necessario poter usufruire di una nozione della bontà di un determinato stato di gioco.

3.1 Esplorazione mosse

3.1.1 Potatura alfa-beta

La potatura alfa-beta è un algoritmo di ricerca che ha l'obiettivo di ridurre il numero di nodi esplorati dall'algoritmo minimax. Interrompe l'esplorazione di un nodo nel momento in cui determina che la mossa in oggetto sia sicuramente peggiore di almeno un'altra mossa precedentemente esplorata. Non è pertanto necessario continuare a esplorare tali mosse.

Negamax

La ricerca **negamax** è una variante dell'algoritmo minimax che sfrutta la proprietà di *somma zero* di giochi come Hive. Semplifica l'implementazione dell'algoritmo minimax basandosi sulla seguente proprietà: $\min(a, b) = -\max(-b, -a)$. Ciò implica che il valore di una posizione del giocatore A è la negazione del valore del giocatore B, che permette di non dover dividere il processo di ricerca in due casi distinti.

Algoritmo Negamax

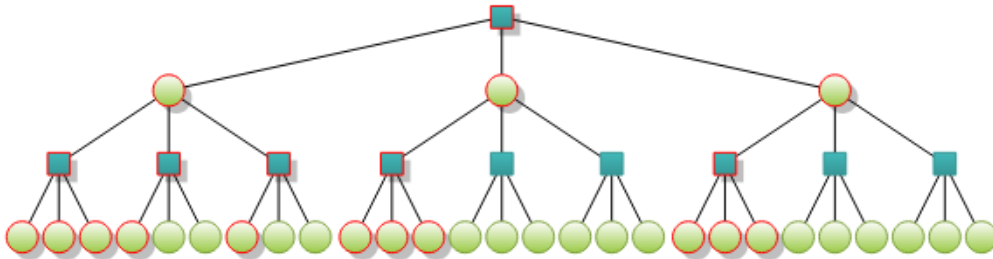
```
function negamax(node, depth,  $\alpha$ ,  $\beta$ , color) is
  if depth = 0 or node is a terminal node then
    return color * the heuristic value of node

  childNodes := generateMoves(node)
  childNodes := orderMoves(childNodes)
  value :=  $-\infty$ 
  foreach child in childNodes do
    value := max(value, -negamax(child, depth - 1,  $-\beta$ ,  $-\alpha$ , -color))
     $\alpha$  := max( $\alpha$ , value)
    if  $\alpha \geq \beta$  then
      break (* cut-off *)
  return value
```

Concetto Young Brothers Wait

Il **concetto Young Brothers Wait** (YBWC) è un elemento fondamentale per la ricerca alfa-beta parallela, coniato da Rainer Feldmann et al. nel 1986 [feldmann].

I fratelli sono nodi che condividono lo stesso nodo padre. Il fratello maggiore, ovvero il primo rispetto ad una determinata funzione d'ordine, è esplorato per primo. I fratelli minori, prima di poter essere esplorati in parallelo, devono attendere che il fratello maggiore ceda il controllo a seguito di un taglio beta nella speranza che riduca i limiti dell'intervallo $[\alpha, \beta]$.



In the best-case alpha-beta game tree, the degree of every node is either 1 or b .

Young siblings wait [FMMV89]

Spawn the first child and immediately sync.

If beta-cutoff, done.

Otherwise, spawn all other children in parallel.

Pray that you don't waste any work.

Figura 1: Slide estratta dalla lezione di M. Frigo "The Cilk system for multi-threading programming"

MTD(f)

MTD(f) è un algoritmo di ricerca alfa-beta con finestra nulla e con tabella di trasposizione per memorizzare i risultati intermedi, creato da Aske Plaatt. Il nome è un'abbreviazione di MTD(n, f) ovvero **Memory-enhanced Test Driver** con nodo n e valore f .

L'efficacia di questo paradigma dipende da una buona stima iniziale del valore finale minimax. In media, migliore è questa supposizione più efficiente sarà l'algoritmo, poiché dovrà svolgere meno passi prima di convergere al valore corretto. Se la stima corrisponde al valore minimax, eseguirà solo due passi: il primo necessario ad individuare il limite superiore del valore x e il secondo per individuare il limite inferiore dello stesso valore.

All'interno del nostro motore di gioco viene utilizzato come stima il precedente valore di valutazione della board della precedente mossa, ad eccezione della prima mossa in cui la stima viene inizializzata a 0.

Algoritmo MTD(f)

```

function MTD(f, d) is
  g := f
  upperBound := +∞
  lowerBound := -∞

  while lowerBound < upperBound do
    β := max(g, lowerBound + 1)
    g := AlphaBetaWithMemory(root, β - 1, β, d)
    if g < β then
      upperBound := g
    else
      lowerBound := g

  return g

```

3.1.2 Monte Carlo Tree Search

MCTS è un algoritmo di **ricerca euristica** utilizzato per l'emulazione di processi decisionali, specialmente all'interno di software in grado di risolvere giochi da tavolo. Nel 2016 è stata per la prima volta dimostrata l'efficacia di MCTS combinato con le reti neurali.

L'algoritmo è basato su esplorazioni randomiche dello spazio di ricerca, a partire dalle quali costruisce gradualmente un albero di gioco e migliora la propria accuratezza.

Le quattro fasi

- **Selezione.** L'albero è esplorato dalla radice R fino a raggiungere una foglia L , dove la radice è lo stato di gioco corrente e una foglia è qualsiasi nodo che ha figli non ancora esaminati.
- **Espansione.** A meno che L non termini il gioco in maniera decisiva, viene espanso un nodo figlio C .
- **Simulazione.** Effettua una partita casuale a partire dal nodo C .
- **Backpropagation.** Utilizza i risultati della partita per aggiornare le informazioni nei nodi sul percorso da C a R .

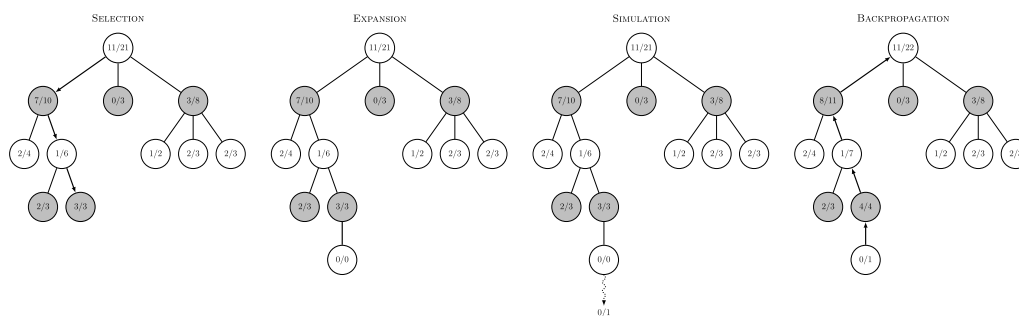


Figura 2: I 4 step della ricerca ad albero di Monte Carlo

3.2 Valutazione stato di gioco

3.2.1 Funzioni euristiche

L'obiettivo è la definizione di una funzione che restituisca una valutazione della bontà (di qui in avanti *score*) del corrente stato dell'alveare. Abbiamo deciso di implementarla mediante la composizione dei seguenti fattori:

- **Valore dei pezzi.** A ogni pezzo è assegnato un valore.
 - **Ape Regina:** 12
 - **Zanzara:** 8
 - **Formica:** 7
 - **Scarabeo:** 6
 - **Coccinella:** 6
 - **Onisco:** 5
 - **Cavalletta:** 3
 - **Ragno:** 2
- **Pezzi non in gioco.** Per ogni diversa creatura viene calcolata la differenza tra i pezzi non in gioco propri e dell'avversario. Questa viene poi moltiplicata per il valore della creatura e il fattore di pezzo non giocato. Viene premiato l'averne meno pezzi in gioco rispetto all'avversario.
- **Pezzi mobili in gioco.** Lo score viene calcolato moltiplicando il valore dei pezzi per il fattore di mobilità delle creature. Inoltre, il valore dei pezzi disposti sopra l'alveare è doppio.
- **Libertà dell'Ape Regina.** Per ogni pezzo adiacente alla propria Ape Regina viene rimosso dallo score il fattore di libertà e, in maniera inversa, per ogni pezzo adiacente all'Ape Regina nemica il fattore viene sommato allo score.
- **Versatilità della Zanzara.** Quando è in gioco, il valore della Zanzara corrisponde alla somma tra il numero di pezzi adiacenti ad essa e il valore massimo tra le creature adiacenti, fatta eccezione per le Api Regina e l'altra Zanzara.
- **Onisco's Law.** Se il proprio Onisco è in grado di spostare la propria Ape Regina in una posizione tale che vi siano più di due spazi liberi adiacenti ad essa, allo score viene sommato il fattore di difesa della Regina. In modo inverso, se l'Onisco avversario è in grado di effettuare la stessa mossa con l'Ape Regina avversaria, il fattore è sottratto allo score. Inoltre, se è possibile piazzare un Onisco vicino all'Ape Regina viene sommato (o, nel caso inverso, sottratto) metà del fattore di difesa allo score.

4 Confronto tra le implementazioni